

FlightRisk

Phase 1 - Exploratory Data Analysis Report

Project objective: estimate flight arrival-delay risk before departure using only information genuinely available at prediction time.

Repository: github.com/Mohammed-Benrebrit/FlightRisk

Dataset used for EDA: Kaggle Flight Delay Dataset - 2024 (10,000-row sample)

Phase status: EDA completed; no trained-model performance is claimed in this report.

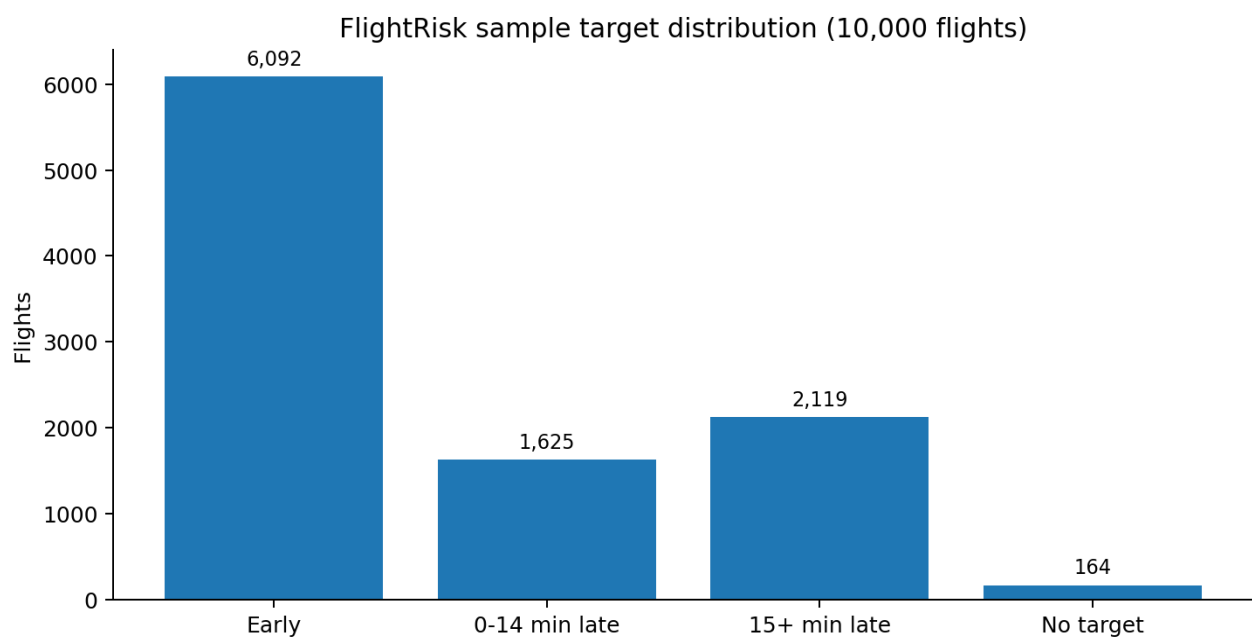
Phase 1 deliverable	Status
Problem framing and prediction contract	Completed
Schema and data dictionary review	Completed
Missing-target investigation	Completed
Leakage audit	Completed
Target distribution analysis	Completed
Candidate feature analysis	Completed
Reusable EDA helper functions	Completed
Transition to modeling dataset preparation	Ready

This report is designed to be committed under **reports/** as evidence of the project's first completed analytical phase.

1. Executive summary

Phase 1 focused on understanding the data and proving that the prediction problem is well-defined before any model is trained. The work followed a leakage-aware workflow: inspect the schema, identify the prediction target, explain missing target values, characterize the target distribution, reject post-departure variables, and study candidate pre-departure features.

Metric	Observed in 10k sample
Rows	10,000 flight records
Raw variables	35
Valid arrival-delay targets	9,836
Cancelled flights	122
Diverted flights	42
Missing arrival-delay values	164 (fully explained by cancelled/diverted flights)
Median arrival delay	-6 minutes
Mean arrival delay	+7.55 minutes
Significant delays	2,119 flights with arr_delay >= 15 min
Maximum observed arrival delay	2,014 minutes



Among valid targets, approximately 61.9% were early, 16.5% were 0-14 minutes late, and 21.5% were at least 15 minutes late.

Key conclusion

The data is suitable for a first supervised-learning baseline, but only after restricting the modeling population to rows with a defined arrival-delay target and excluding all information that becomes available during or after the flight.

2. What the EDA phase covered

Workstream	What was done	Why it matters professionally
Data discovery	Located sample, full data, and data dictionary; inspected shape and schema.	Prevents assumptions about files, fields, and memory footprint.
Schema understanding	Reviewed variable names, data types, examples, and missingness.	Establishes a reliable data contract before modeling.
Target validity	Traced missing arr_delay to cancellations and diversions.	Avoids inventing labels or training on undefined outcomes.
Leakage audit	Separated schedule-time features from actual departure, airborne, landing, arrival, and delay-cause information.	Protects evaluation from unrealistically strong results.
Target analysis	Measured counts, mean, median, percentiles, skew, and extreme values; visualized distributions.	Shows class balance and regression-target difficulty.
Feature investigation	Compared carrier, departure hour, calendar fields, airports, distance, and scheduled duration against delay behavior.	Builds evidence-based candidate features instead of blindly using all columns.
Code quality	Refactored repeated grouped-analysis and plotting logic into reusable functions.	Demonstrates maintainability beyond one-off notebook code.

Professional note

This is a solid project-level EDA workflow. A production team would usually add stronger automation around schema validation, data lineage, duplicates, full-dataset profiling, train-only EDA after temporal splitting, monitoring/drift checks, and formal tests for recurring pipelines. University work may add more explicit statistical justification and methodology. The principles used here - target definition, missingness reasoning, leakage prevention, robust statistics, and documented feature hypotheses - are directly transferable.

3. Target contract and leakage policy

Regression target: arrival delay in minutes (**arr_delay**). Negative values represent early arrival.

Classification target: 1 when arr_delay >= 15 minutes, otherwise 0, for rows where arr_delay is defined.

Prediction time: before departure. This is the rule that determines whether a variable may become a feature.

Category	Examples	Decision
Pre-departure candidates	op_unique_carrier, origin, dest, scheduled times, calendar fields, distance, crs_elapsed_time	Eligible for evaluation
Post-departure leakage	dep_time, dep_delay, taxi_out, wheels_off, wheels_on, taxi_in, arr_time, air_time, actual_elapsed_time	Do not use as model inputs
Outcome-attribution leakage	carrier_delay, weather_delay, nas_delay, security_delay, late_aircraft_delay	Do not use as pre-departure inputs
Undefined-target cases	cancelled or diverted flights with missing arr_delay	Exclude from current arrival-delay targets; preserve for future disruption models

4. EDA findings and candidate features

Variable / engineered feature	EDA evidence	Current status
departure_hour	Clear increase in mean delay / 15+ minute delay rate from morning into afternoon/evening; very sparse overnight hours require caution.	Strong candidate
day_of_week	Meaningful differences between weekdays; not a simple linear trend.	Moderate/strong candidate
month	Clear seasonal variation in sample delay behavior.	Moderate/strong candidate
op_unique_carrier	Large differences in mean, median, and 15+ minute delay rate between carriers.	Strong candidate
origin	Airport-level variation after filtering low-count categories.	Useful categorical candidate
dest	Destination-level variation after filtering low-count categories.	Useful categorical candidate
crs_elapsed_time	Weak-to-moderate relationship; extreme duration buckets have very low counts. Preserve original minutes for modeling.	Keep initially
distance	No clear monotonic relationship in scatter plot, but may interact with route/carrier/time.	Keep initially; weak alone
day_of_month	No stable pattern in the 10k sample.	Weak candidate / optional

Important modeling distinction

EDA buckets are not automatically model features. For example, grouping scheduled elapsed time into whole hours is useful for visualization, but the original **crs_elapsed_time** in minutes preserves more information for modeling. Likewise, day-of-week and airport codes are categorical concepts even if they are stored as integers or strings.

5. Engineering and learning outcomes

Skill evidenced	Evidence from Phase 1
pandas	Data loading, filtering, boolean masks, null analysis, groupby, aggregation, quantiles, sorting, categorical counts.
NumPy / numerical reasoning	Boolean-to-rate logic, distribution reasoning, feature transformations, careful interpretation of numeric encodings.
Matplotlib	Bar charts, histograms, scatter plots, ordered trend plots, axis-range control for skewed targets.
ML problem framing	Regression and classification targets defined before training.
Leakage prevention	Prediction-time availability used as the feature-eligibility rule.
Feature engineering	Scheduled HHMM departure time converted into departure hour for analysis.
Statistical thinking	Mean vs median, skew, percentiles, tail behavior, sparse-group caution, rates normalized by sample size.
Software engineering	Repeated EDA logic extracted into reusable summarization and plotting functions.

Recruiter value

The strongest signal is not the number of plots. It is the reasoning visible behind them: defining a prediction contract, testing hypotheses, detecting leakage, validating missingness, documenting uncertainty, and refactoring repeated analysis. To make this job-relevant, the repository should preserve that evidence through a clean

notebook, this report, reproducible local setup, clear commits, and later tests/pipelines.

6. GitHub repository audit and recommended update

Repository reviewed: **Mohammed-Benrebrit/FlightRisk**. At the time of this report the root contains **.gitignore**, **README.md**, and **PROJECT_PLAN.md**. The written plan is strong, but the repository still describes data exploration as in progress and does not yet contain the completed notebook/report artifacts.

Current strength	Recommended update after Phase 1
README defines prediction time, regression/classification goals, and leakage policy.	Change status from EDA in progress to Phase 1 EDA completed; add concise findings and link to the report/notebook.
PROJECT_PLAN has leakage rules, chronological validation intent, baseline-first modeling, and future engineering phases.	Mark completed data-understanding items and align phase numbering/status with actual work.
.gitignore already excludes local raw/interim/processed data, models, artifacts, virtual environments, and notebook checkpoints.	Keep this policy; do not commit the raw Kaggle dataset. Add acquisition instructions instead.
Repository does not yet contain the EDA notebook or reports directory.	Add notebooks/01_eda.ipynb and reports/FlightRisk_Phase1_EDA_Report.pdf once reviewed.

Suggested repository state after committing Phase 1

```
FlightRisk/
|-- README.md
|-- PROJECT_PLAN.md
|-- notebooks/
| `-- 01_eda.ipynb
|-- reports/
| `-- FlightRisk_Phase1_EDA_Report.pdf
|-- src/ (introduced as Phase 2 code becomes reusable)
|-- tests/ (introduced with reusable feature/data logic)
`-- data/README.md (dataset source + acquisition notes; raw data stays ignored)
```

7. Recommended transition to Phase 2

Recommendation: move the project locally now. Kaggle was the right sandbox for learning the dataset. From Phase 2 onward, the work should live primarily inside the GitHub repository using VS Code + a local Jupyter kernel. Kaggle remains useful as a data source, compute fallback, and reproducibility reference.

Environment	Role from Phase 2 onward
VS Code + local Jupyter	Primary development environment: notebook experiments, Git commits, refactoring, tests, and project structure.
GitHub	Source of truth for code, documentation, reports, issues, and history.
Kaggle	Dataset acquisition / exploration fallback / optional full-data compute; not the main project workspace.
Local data directories	data/raw, data/interim, data/processed remain untracked; store acquisition instructions in version control.

8. Recommended Phase 1 commit checklist

Repository change	Purpose
Add notebooks/01_eda.ipynb	Preserve the complete exploratory workflow and reasoning as an executable artifact.
Add reports/FlightRisk_Phase1_EDA_Report.pdf	Provide a concise, recruiter-friendly summary of the completed phase.
Update README.md	Mark EDA complete, summarize findings, and link to notebook/report.
Update PROJECT_PLAN.md	Check completed data-understanding tasks and align phase status with actual work.
Add data/README.md	Document Kaggle dataset identifier, expected files, and acquisition steps without committing raw data.

Immediate Phase 2 objective

Create a clean modeling dataset while preserving raw inputs. Define the valid modeling population, choose leakage-safe pre-departure variables, create the engineered features needed by the first baseline, and only then construct X and y. Chronological splitting and preprocessing must follow before model evaluation.

End of Phase 1

Phase 1 is now a defensible completed milestone: the problem is framed, target validity is understood, leakage risks are documented, the target and candidate features have been explored, and the project is ready to transition from exploratory notebook work to reproducible modeling engineering.